Group Project: Building an End-to-End CI/CD Pipeline

1. Project Introduction

This is the group project for the DevOps Fundamentals course. The goal of this project is not to build a complex web application, but rather to build a professional, automated pipeline around a provided web application.

You will apply all the skills you've learned (**Git, Docker, GitHub Actions, Docker Compose, Kubernetes** concepts) to create a complete **CI/CD** (Continuous Integration / Continuous Deployment) process, starting from a git commit on a developer's machine to a live application running on a production server.

Focus: The automation process, the quality of the pipeline, stability of the workflow, and understanding when to scale to Kubernetes - not the features of the web application.

2. Group Requirements

- **Size:** 2-3 students per group.
- **Roles:** Each group must designate **1 member** as the *Team Leader*.
- **Team Leader Responsibilities:**
  - Fork the "*Base Project*" to their personal GitHub account.
  - Rename the repository to the group's name (e.g., `devops-group-1`).
  - Manage the repository: Invite team members (and the lecturer) as "*Collaborators*" and be the final person responsible for merging Pull Requests.

3. Base Project

You will be given a GitHub repo link to a base web project (a simple To-Do list app using React - Frontend, Node - Backend and PostgreSQL - Database).

- This application is **intentionally simple**.
- It contains a `package.json` file with defined scripts (like `"start"`, `"test"`).
- It **contains intentional bugs** in the code and test cases that you must identify and fix.
- The ***Dockerfiles*** and `docker-compose.yml` are provided as **skeletons** – you must complete them.
- Your task is **not** to add new features, but to "*wrap*" this application in a professional DevOps process
- GitHub Repo link: https://github.com/longndt/devops-project

🚨 **Known Issues in Base Project (You Must Fix):**

- Backend `server.js`: 6 bugs marked with comments
- Tests: 4 out of 7 tests will fail until bugs are fixed
- **Dockerfiles**: Only *skeleton/comments* provided
- `docker-compose.yml`: Incomplete configuration
- Missing `.dockerignore` files
- No `.github/workflows/` directory

4. Technical Requirements

A. Git & Collaboration

- All project code must be stored in a single, shared group GitHub repository.
- The `main` branch must be protected. No one (including the *Team Leader*) is allowed to push directly to `main`.

- All changes (code, config) must be made on **feature branches** (e.g., `feature/add-dockerfile`, `ci/setup-workflow`).
- All feature branches must be merged into `main` via a **Pull Request (PR)**.
- Every PR **must** be reviewed and "**Approved**" by at least **one other team member** before being merged.
- All commit messages must follow the **Conventional Commits** standard (e.g., `feat:`, `fix:`, `ci:`, `docs:`).

B. Containerization

- Each service must have its own **Dockerfile**:
  - `backend/Dockerfile` (**Node** backend ⇨ node:18-alpine)
  - `frontend/Dockerfile` (**React** frontend ⇨ node:18-alpine)
  - Both use **multi-stage build** (builder + production stages)
  - The final images must be small (e.g., based on `node:18-alpine`).
- A `.dockerignore` file for both frontend/backend (exclude `node_modules`, `.git`, `.env`).
- A `docker-compose.yml` file must be in the project root.
  - 3 services: **frontend (***port 3000***), backend (***port 8080***), postgres** (*port 5432*).
  - Backend env vars: **DB_HOST=postgres, DB_USER=postgres**, etc.
  - Named volume: **`postgres_data`**
  - Healthchecks for all services
  - Mount DB init script: **`./db/init.sql:/docker-entrypoint-initdb.d/init.sql`**

C. Continuous Integration (CI)

- A workflow file (`.github/workflows/ci.yml`) triggered on **pull request** to main branch
- The workflow must be triggered every time a **Pull Request** is opened or updated targeting the `main` branch.

- The `build-and-test` job must launch a database service (using `services:`) to run tests in an integrated environment.
- The pipeline must execute these steps:
  1. `actions/checkout@v4`
  2. `actions/setup-node@v4 (node:18)`
  3. `npm ci`
  4. `npm test` (`DB_HOST=localhost:5432`, `DB_USER`, etc.)
  5. Build **both** frontend/backend Docker images
  6. Push to Docker Hub
     Ex: `<username>/devops-group1-frontend`, `<username>/devops-group1-backend`
- Docker Hub secrets: `DOCKER_HUB_USERNAME, DOCKER_HUB_PASSWORD`

D. Continuous Deployment (CD)

- Add a second job named `deploy` to the `ci.yml` file.
- The `deploy` job **must only** run if two conditions are met:
  1. The `build-and-test` job has succeeded (use `needs: build-and-test`).
  2. The event is a `push` to the `main` branch (use `if: github.ref == 'refs/heads/main'`).
- The deploy job **must** use **GitHub Secrets** to store server credentials (`HOST`, `USERNAME`, `SSH_PRIVATE_KEY`).
- The deploy job must use an SSH action (e.g., `appleboy/ssh-action`) to connect to the server.
- The deploy script on the server must execute (at minimum) these commands:

```
cd /path/to/your-project

git pull origin main

docker-compose pull

docker-compose up -d
```

E. Kubernetes Understanding

- In your presentation, **explain**:
    1. Current limitations of Docker Compose (single server, no auto-scaling).
    2. When your project would need Kubernetes (100+ containers, multiple servers, auto-scaling).
    3. Core Kubernetes concepts: **Pod** (1 app instance), **Deployment** (manages 3+ pods), **Service** (stable access point).

5. Final Presentation

- **Format:** Group Presentation (Live Demo).
- **Time:** 10-15 minutes per group (approx. 10 min demo + 5 min Q&A).
- **Deliverables:**
    1. A complete slide desk following below structure
    2. A clear report which illustrates the project development process
    3. A **LIVE DEMO** of the complete pipeline using "**Fail-To-Fix**" flow
- **Presentation Structure:**
    1. Project Overview
    2. Architecture & Design
    3. Containerization (Docker + Compose)
    4. CI/CD Pipeline
    5. Deployment Strategy
    6. Challenges & Kubernetes Migration Plan
    7. Q&A
- **Report Structure:**
    1. **Team & Repository**
        - Team members & GitHub usernames
        - Repository: **[link]**

        - Branch protection: **[screenshot]**
    2. **Development Process**
        - Total PRs created
        - Total commits
        - Challenges faced & solutions
    3. **Technical Decisions**
        - Why multi-stage Dockerfile?
        - CI triggers explanation
        - CD deployment strategy
    4. **Pipeline Metrics**
        - CI runtime
        - CD deployment time
        - Test coverage
        - Docker image size
    5. **Kubernetes Migration Plan**
        - Current: Docker Compose (1 server)
        - Migrate when: 100+ containers, auto-scaling
        - K8s objects needed: Deployment + Service
    6. **Learnings & Next Steps**
        - Biggest challenge solved
        - DevOps principles (CALMS) applied
        - Future improvements
- **Project Submission:**
    - Push to GitHub repository (counts as collaboration proof)

6. Assessment Criteria

1. Slide (30%)

- Architecture diagram
- CI/CD workflow explanation
- Docker + Kubernetes concepts

2. Report (30%)

- Process documentation
- Technical decisions justifed
- Metrics & Kubernetes plan

3. Presentation + Q&A (40%)

- Fail-Fix-Pass demo session
- Technical Q&A competence

7. Demo Flow ("Fail-Fix-Pass" Flow)

1. **Show live app :** http://your-server-ip-address *(frontend)*
2. **Break backend test** ⇨ **PR** ⇨ **CI fails** (Red ✗)
3. **Fix** bug ⇨ **Push** ⇨ **CI passes** (Green ✓)
4. **Approve + Merge** PR ⇨ **CD runs** automatically
5. Show **Actions** tab: **build-and-test** ⇨ **deploy** jobs
6. **Refresh** browser ⇨ New version **live**